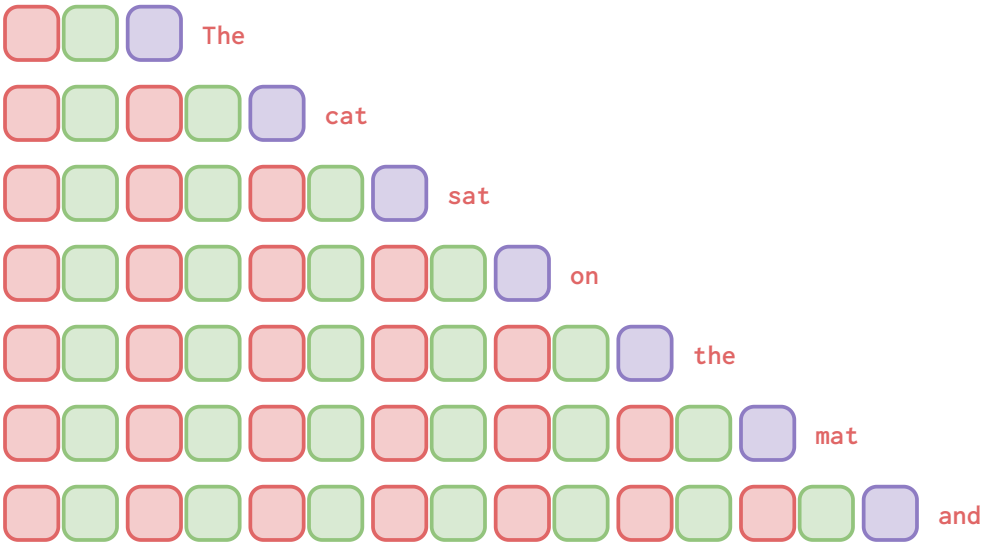


UNIT 01 · CHAPTER 10

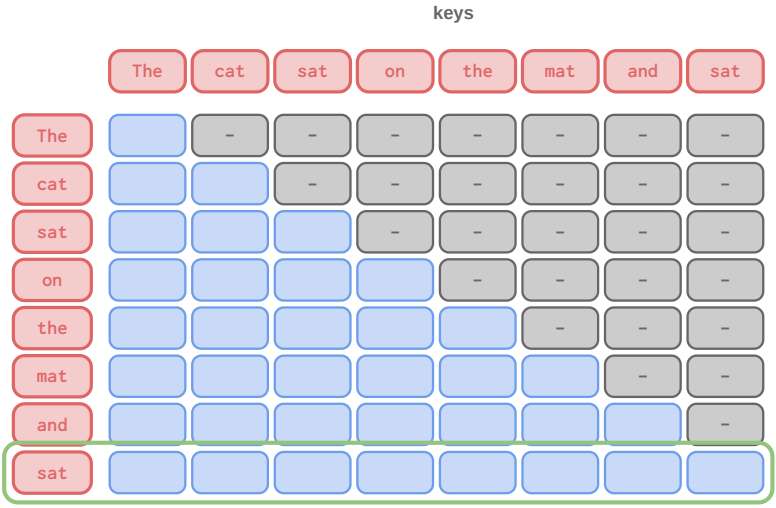
Attention Variants

CSE 10124 · Building ChatGPT
William Theisen · University of Notre Dame

Auto-Regressive Generation

Multi-head attention gives every token in "The cat sat on the mat and" a context built from every token before it, and the context of the last token is what the model uses to predict the next one. Say it predicts "sat" (the cat apparently sits twice in this story).

To keep writing, we stick "sat" on the end of the prompt and run the whole model again on all eight tokens, then again on nine, and so on. Feeding a model its own output one token at a time like this is called **auto-regressive generation**, and it's how every chatbot writes its answers.



Pictured above is the attention table for the second pass. Notice that the first seven rows are exactly the table from the first pass. Each token can only look backwards, and nothing before "sat" has changed, so **the only new row is the one for "sat"**.

Repeated Work

So on every step we recompute every row, keep the last one, and throw the rest away. How much is that? Each pass computes a key and a value for every token it's given. Starting from our `7`-token prompt and generating `20` more tokens, the passes see `7`, `8`, `9`, … `26` tokens, `330` keys and values in all. Only `26` of them were ever new.

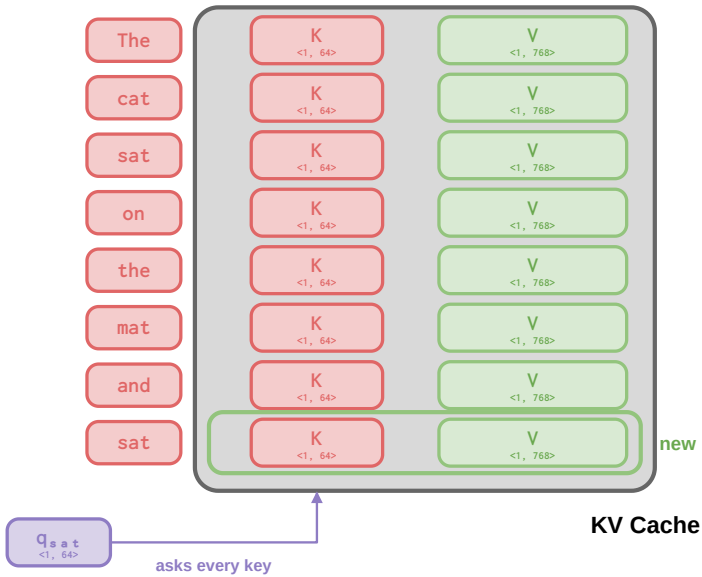
It gets worse the longer the conversation. Writing `4,096` tokens one at a time, starting from a single token, works out `8,390,656` keys, when `4,096` would have done. **The waste grows with the square of the length**, the same way the attention table itself does.

Of course the model does need that last row, so the real question is what the last row needs, and which of those things we already have.

The KV Cache

The new row needs three things: the query for "sat", and the keys and values of all eight tokens. The query is new. The keys and values of the seven old tokens, though, are exactly what we computed last time, since a token's key is just its embedding times W_K and neither of those has changed.

So we keep them. A **KV cache** is a store of every key and value we've already computed, one pair per token, so that **each new token only computes its own key and value and looks up the rest**.



Attention with a Cache

Each step now computes one query, one key and one value, adds the key and value to the cache, and does attention for a single row:

$$\texttt{Context}_{\texttt{sat}} = \texttt{softmax}\left(\frac{q_{\texttt{sat}} K^{\top}}{\sqrt{d_k}}\right) V$$

Here K and V are the eight cached keys and values. In multi-head attention every head has its own keys and values, so every head has its own cache (and so does every layer).

A KV Cache in Code

We can check that the cache really gives the same answer. Here is one head at the slides' sizes, with random numbers standing in for our eight tokens' embeddings:

```python
import torch
torch.manual_seed(0)
d, dk = 768, 64
E = torch.randn(8, d)     # The cat sat on the mat and sat
Wq = torch.randn(d, dk) / d**0.5
Wk = torch.randn(d, dk) / d**0.5
Wv = torch.randn(d, d) / d**0.5

# Without a cache: all 8 rows again, keep the last one
Q, K, V = E @ Wq, E @ Wk, E @ Wv
future = torch.triu(torch.ones(8, 8), diagonal=1) == 1
S = (Q @ K.T / dk**0.5).masked_fill(future, float('-inf'))
full = torch.softmax(S, dim=-1) @ V

# With a cache: K and V of the 7 prompt tokens were kept
K_cache, V_cache = E[:7] @ Wk, E[:7] @ Wv
e = E[7:]                    # only "sat" is new
K_cache = torch.cat([K_cache, e @ Wk])
V_cache = torch.cat([V_cache, e @ Wv])
a = torch.softmax((e @ Wq) @ K_cache.T / dk**0.5, dim=-1)
cached = a @ V_cache
print(a.shape, cached.shape)
print((cached - full[7:]).abs().max())
```

OUTPUT

```
torch.Size([1, 8]) torch.Size([1, 768])
tensor(4.7684e-07)
```

The two agree to within 4.8×10^{-7}, which is just rounding (the sums happen in a different order). So the cache saves all that work without changing the answer.

Prefill and Decode

With a cache, generating text splits into two quite different phases. First the model reads the whole prompt at once, all seven tokens in parallel, which fills the cache with their keys and values. This is called **prefill**. Then it writes the answer one token at a time, each step adding one key and one value per head per layer. This is **decode**.

irishQwen keeps one small `SimpleKVCache` per transformer block (filling in its `append` is one of the exercises), and its `chat` method works exactly like this: one call to `prefill` with the prompt, then a `decode_step` for every new token, passing along how many tokens came before it so the model knows where in the sequence the new one sits.

The two phases are hard work for a GPU in different ways. Prefill is a few big matrix multiplications, which GPUs are very good at. Decode does very little math per step but has to read the whole cache every step, so **the bigger the cache, the slower every new token**. Which raises the question of how big it actually gets.

The Cost of the Cache

Unfortunately the cache isn't free. Every token keeps a key and a value for every key head in every layer. If Qwen3-0.6B gave each of its **16** heads its own keys and values, like the multi-head attention from the last chapter, that would be $2 \times 16 \times 128 = 4{,}096$ numbers per token per layer, and **114,688** numbers per token across its **28** layers:

```
def cache_gb(tokens, kv_heads, layers=28, dh=128):
    numbers = 2 * kv_heads * dh * layers * tokens
    return numbers * 4 / 1e9   # float32: 4 bytes each

for T in [7, 4096, 32768]:
    print(T, "tokens:", round(cache_gb(T, 16), 3), "GB")
```

OUTPUT

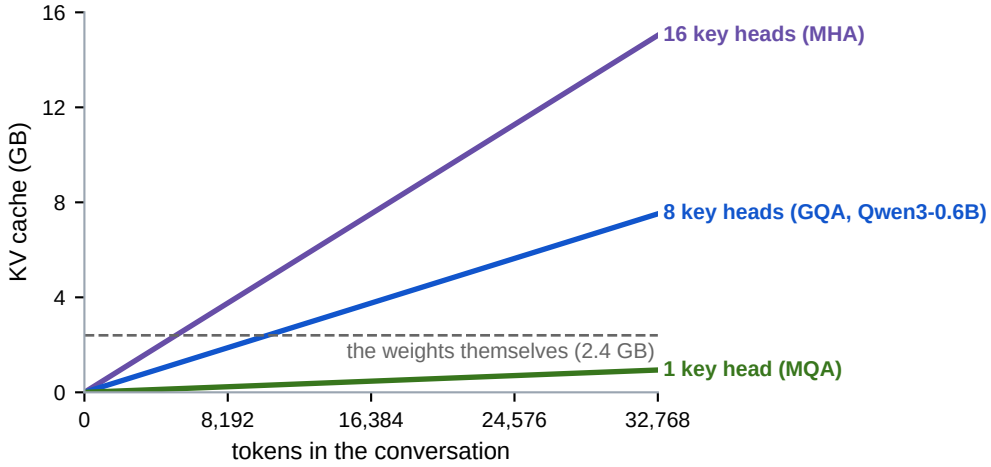
```
7 tokens: 0.003 GB
4096 tokens: 1.879 GB
32768 tokens: 15.032 GB
```

irishQwen keeps every number as a **4**-byte float, so a full **32,768**-token conversation (Qwen3-0.6B's context length) would need **15** GB of cache. The model's weights are only about **2.4** GB (**0.6** billion numbers). **Remembering the conversation would take more than six times the memory of the model itself.**

The cache is keys and values times heads times layers times tokens. The tokens are the conversation, and the layers are the model, so the obvious place to start cutting is the heads.

Multi-Query Attention

Every query head needs its own query, that's what lets the heads look for different things. But does each one need its own keys and values? Shazeer (2019) tried the extreme answer, all the query heads sharing a single key head and a single value head. This is **multi-query attention** (MQA), and it divides the cache by the number of heads, from **15** GB to under **1** GB in our example.

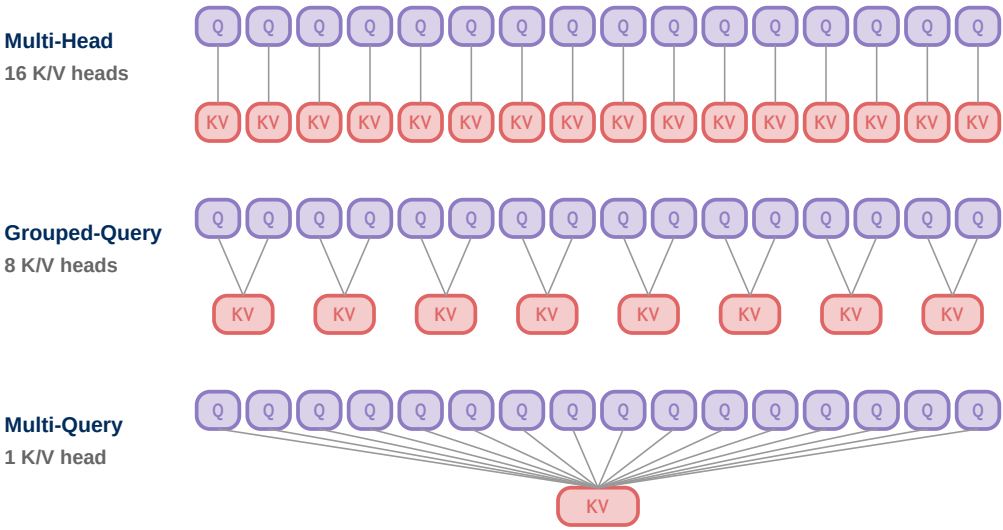


Pictured above is the cache for Qwen3-0.6B's shape with **16**, **8** and **1** key heads. All three grow in a straight line with the conversation, only the slope changes.

Unfortunately, sharing one set of keys costs some quality. The heads can still ask different questions, but they're now all matching them against the same keys and getting back the same values.

Grouped-Query Attention

The middle ground is to share keys and values within small groups of query heads. Ainslie et al. (2023) called this **grouped-query attention** (GQA). With **16** query heads and **8** key/value heads, each pair of query heads shares one key head and one value head, which is exactly what Qwen3-0.6B does.



Multi-head and multi-query attention are really just the two ends of GQA, with groups of one and one group of everything. Qwen3-0.6B's groups of two halve its cache, from **15** GB to **7.5** GB at **32,768** tokens (the blue line on the last page).

Halving doesn't sound like much, but it's cheap: the GQA paper found that grouped heads get close to multi-head quality at close to multi-query speed, and most of the models later in this chapter use it.

Grouped-Query Attention in irishQwen

irishQwen's `GroupedQueryAttention` builds **16** query heads but only **8** key heads and **8** value heads (its `Wq` has **2,048** rows, `Wk` and `Wv` have **1,024**). Only the **8** key and value heads go in the cache.

When the time comes to compare queries with keys, it copies each key head once for every query head in its group, with `repeat_interleave`:

```
import torch
K = torch.randn(1, 8, 7, 128)    # 8 key heads, 7 tokens
K_rep = K.repeat_interleave(2, dim=1)
print(K.shape, "->", K_rep.shape)
# query heads 4 and 5 both read key head 2
print(bool(torch.equal(K_rep[0, 4], K[0, 2])),
      bool(torch.equal(K_rep[0, 5], K[0, 2])))
```

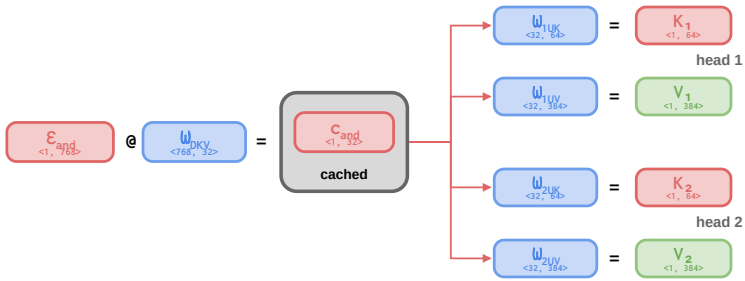
OUTPUT

```
torch.Size([1, 8, 7, 128]) -> torch.Size([1, 16, 7, 128])
True True
```

Query heads **4** and **5** both read key head **2**. The copies are only made for the multiplication, the cache keeps the **8** originals, which is where the saving comes from.

Multi-Head Latent Attention

Sharing keys and values is one way to shrink the cache. DeepSeek went another way with **multi-head latent attention** (MLA). Instead of storing keys and values at all, each token stores one short vector, called a latent, and every head rebuilds its own keys and values from it when it needs them.



In the slides' toy layer, W_{DKV} squeezes each **768**-number embedding to **32** numbers, and each head's W_{UK} and W_{UV} turn those back into a **64**-number key and a **384**-number value. Two heads keeping their own would cache $2 \times (64 + 384) = 896$ numbers per token, the latent is **32**.

Real models save even more. DeepSeek-V3 has **128** heads of **128** numbers, so plain multi-head attention would cache **32,768** numbers per token per layer, and MLA caches **576** (a **512**-number latent plus a **64**-number key that carries position, which the trick on the next page can't handle). DeepSeek reported that MLA cut the cache of DeepSeek-V2 by **93.3%** compared with their earlier 67B model.

Absorbing the Weights

Rebuilding every key on every step would undo the saving, since we'd be back to computing a full key for every cached token. It turns out we never have to. Here is head `i`'s query meeting its rebuilt keys:

$$
\begin{aligned}
q K^{\top} &= (\varepsilon W_{CQ})(C W_{UK})^{\top} \\
&= \varepsilon W_{CQ} W_{UK}{}^{\top} C^{\top} \\
&= \varepsilon W'_{CQ} C^{\top}
\end{aligned}
$$

The middle line uses $(AB)^{\top} = B^{\top} A^{\top}$. W_{CQ} and $W_{UK}{}^{\top}$ are both just weights, so we can multiply them together once, before the model is ever used, into one $\langle 768, 32 \rangle$ matrix W'_{CQ}. The query then lands directly in the **32**-number latent space and is compared with the cached latents C themselves. The same trick folds W_{UV} into the output weights, so the values are never rebuilt either. This is called **absorbing** the matrices.

Absorbing in Code

We can check the absorbing with the toy layer's sizes and random numbers standing in for the seven embeddings:

```python
import torch
torch.manual_seed(0)
d, dc, dk = 768, 32, 64
E = torch.randn(7, d)   # The cat sat on the mat and
W_dkv = torch.randn(d, dc) / d**0.5
W_cq = torch.randn(d, dk) / d**0.5
W_uk = torch.randn(dc, dk) / dc**0.5

C = E @ W_dkv           # the latent cache <7, 32>
K = C @ W_uk            # keys rebuilt <7, 64>
scores = (E @ W_cq) @ K.T

W_cq2 = W_cq @ W_uk.T  # absorbed <768, 32>
scores2 = (E @ W_cq2) @ C.T   # never builds K
print(C.shape, W_cq2.shape)
print((scores - scores2).abs().max())
```

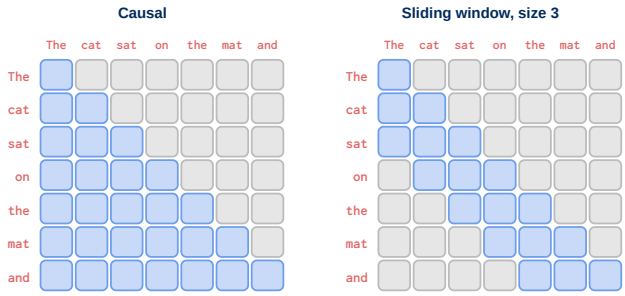
OUTPUT

```
torch.Size([7, 32]) torch.Size([768, 32])
tensor(1.2755e-05)
```

The two ways of getting the scores agree up to rounding, and the second never builds a key. At decode time the new token's query is **32** numbers long and is compared directly with the **32**-number latents in the cache.

Sliding Window Attention

Everything so far shrinks what each token stores. The other way to shrink the cache is to store fewer tokens. In **sliding window attention** (SWA), each token only attends to the last few tokens (the window) instead of all of them, so anything older than the window can be dropped from the cache.



With a window of `3`, "mat" can see "on", "the" and itself but not "The", "cat" or "sat". If every layer did that, the model would forget everything a few tokens back, so models mix them. Gemma 3 uses a window of `1,024` tokens with `5` sliding window layers for every global layer that sees everything. (Older information still arrives indirectly, carried by other tokens.)

The global layers still need the full cache, but the sliding window ones never keep more than `1,024` tokens. At `32,768` tokens, a group of six such layers keeps
`(5 × 1,024 + 32,768) / (6 × 32,768)` = 19% of the cache six global layers would.

Sparse Attention

A window assumes the useful tokens are the recent ones, which often isn't true (a question asked at the start of a long document, say). **DeepSeek Sparse Attention** (DSA) picks the tokens instead. A small, cheap scoring network, which DeepSeek calls the lightning indexer, gives every earlier token s an importance score for the current token t, and the real attention only looks at the k highest:

$$I_{t,s} = \sum_{j} w_{t,j}\,\mathrm{ReLU}\left(q_{t,j} \cdot k_{s}\right)$$

Here $q_{t,j}$ and $w_{t,j}$ come from the current token (one of each per indexer head j) and k_s from the earlier one. The indexer is kept cheap, with fewer heads than the real attention and numbers stored in only **8** bits. In DeepSeek-V3.2 each new token attends to its top **2,048** tokens, however long the conversation gets.

Notice that this saves work rather than memory. Every token still has to be kept, since any of them might make somebody's top **2,048** later, and the indexer keeps small keys of its own on top. DeepSeek-V3.2 uses DSA together with MLA, so its cache was already small.

Gated Attention

Qwen went further still and replaced most of its attention layers with something that has no growing cache at all. Before getting to that, there's one small change to the attention layers they kept. In **gated attention**, the output of each head is multiplied, number by number, by a gate between `0` and `1` worked out from the token itself:

$$\text{Context}' = \text{Context} \odot \sigma(\mathcal{E} W_G)$$

The sigmoid lets each token turn parts of its attention output down, or off. Qiu et al. (2025) tried gates in several places and found this one (right after the attention, before the output weights) worked best. It also got rid of the "attention sink", the habit attention layers have of dumping most of their weight on the first token when there's nothing better to look at.

Qwen3-Next uses gated attention for every attention layer it keeps. What it uses for the rest goes back to an idea from a few chapters ago.

A Fixed-Size Memory

Recurrent neural networks never had a cache problem. An RNN carries one fixed-size vector h from token to token, so the thousandth token costs exactly as much as the first. Their problem was the opposite one, everything had to squeeze through that single vector, and older tokens faded out of it.

Gated DeltaNet (Yang, Kautz and Hatamizadeh, 2025) keeps the fixed-size state but makes it much bigger and much more organized. Each head carries a matrix S (64×64 in the slides' toy head) that stores key → value pairs. Multiply a key by S and out comes (approximately) the value that was stored with it:

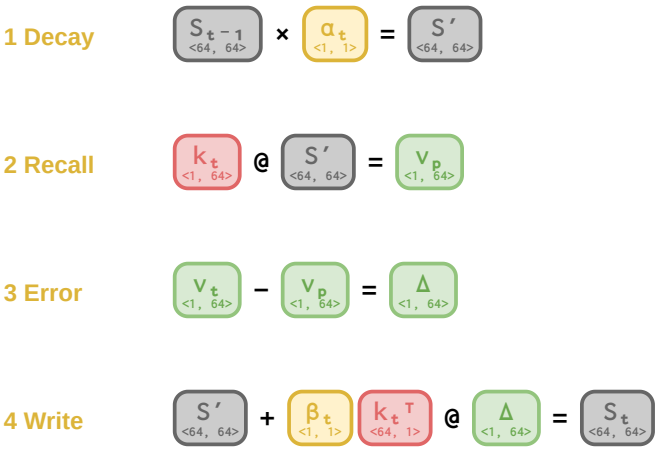
$$k\,S \approx v$$

That's the same question attention asks (which value goes with this key?). The difference is that instead of keeping every key and value and comparing against all of them, all the pairs are written into one matrix that never grows. At 64×64 that's **4,096** numbers per head, whether the conversation is **7** tokens long or **32,768**.

Reading works the same way: the new token multiplies its query by the memory, $o_t = q_t\,S_t$. There's no softmax anywhere, which makes this a kind of linear attention.

The Delta Rule

Writing into S is where the name comes from. For each new token, Gated DeltaNet takes four small steps:



First it shrinks the whole memory a little (α_t is between `0` and `1`, so old pairs slowly fade). Then it asks the memory what it currently holds for this token's key, compares that with the token's actual value, and writes back only the difference Δ, scaled by β_t. Writing the correction rather than the value itself is the **delta rule**, and it means a pair that's already stored correctly barely changes the memory at all.

α_t and β_t are gates worked out from the token, so the model decides token by token how much to forget and how much to write.

The Delta Rule in Code

Here are three key → value pairs written into an empty 64 × 64 memory with the four steps (no forgetting, α = 1, and full writes, β = 1), then read back out, printing how far off each one comes back:

```python
import torch
torch.manual_seed(0)
keys = torch.randn(3, 64)
keys = keys / keys.norm(dim=1, keepdim=True)
values = torch.randn(3, 64)

S = torch.zeros(64, 64)        # the state
alpha, beta = 1.0, 1.0
for k, v in zip(keys, values):
    S = alpha * S              # 1 decay
    v_p = k @ S                # 2 recall
    delta = v - v_p            # 3 error
    S = S + beta * torch.outer(k, delta)  # 4 write

for k, v in zip(keys, values):
    err = (k @ S - v).norm() / v.norm()
    print(round(err.item(), 3))
```

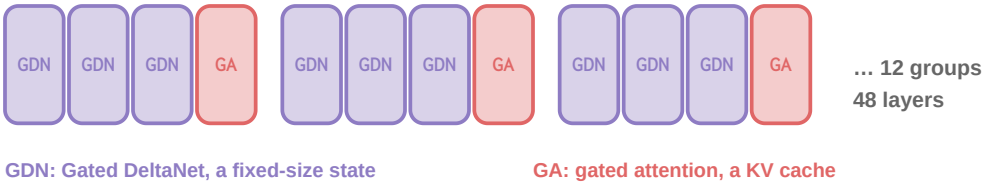
OUTPUT

```
0.197
0.138
0.0
```

The last pair comes back exactly, the first two come back about **20%** and **14%** off. Each write corrects the memory for its own key, but random keys are never quite perpendicular, so it also nudges what the memory returns for the others. **A fixed-size memory can only hold so much**, which is why the read is only "approximately" the value.

Hybrid Attention

So Gated DeltaNet is cheap and never grows, but it's worse than softmax attention at fetching one exact token from long ago. Qwen3-Next's answer is not to choose, and to stack both kinds of layer: three Gated DeltaNet layers, then one gated attention layer, repeated. Its **48** layers are **36** Gated DeltaNet layers and **12** attention layers, and Qwen3.5 keeps the same pattern. This is called **hybrid attention**.



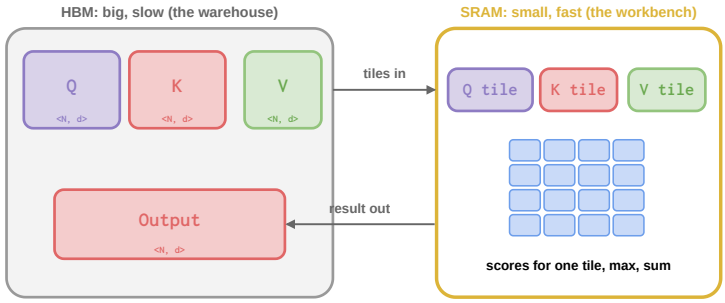
Only the attention layers keep a KV cache, the others carry their fixed-size states. If Qwen3-0.6B's **28** layers were split the same way, its **7.5** GB cache at **32,768** tokens would drop to **1.9** GB (a quarter), plus the DeltaNet states, which don't grow.

The attention layers are kept for what Gated DeltaNet is bad at, finding one exact token from long ago, and the DeltaNet layers do the rest cheaply. **The hybrid gets most of the memory saving and keeps most of the recall.**

Flash Attention

Every variant so far changes what attention computes. **Flash attention** (Dao et al., 2022) computes exactly the same thing, just faster, and works alongside any of them.

The problem it fixes is moving numbers around. A GPU has a huge but slow main memory (HBM, the warehouse) and a tiny but very fast memory next to its compute units (SRAM, the workbench). Plain attention works out the whole score table, writes it to the warehouse, reads it back for the softmax, writes the weights, and reads them back again to multiply by V. For one head at $32{,}768$ tokens that table is $32{,}768 \times 32{,}768$ numbers, 4.3 GB in float32 (69 GB for Qwen3-0.6B's 16 heads). Most of the time goes on the trips, not the math.



Working in Tiles

Flash attention never builds the table. It brings a tile of queries and a tile of keys and values to the workbench, works out that tile's scores, folds them into the answer and moves on, so the table only ever exists one tile at a time. The paper reported GPT-2 training **3×** faster, and since memory now grows with the length of the input rather than its square, much longer inputs fit.

The catch is the softmax. To turn a score into a weight we divide by the sum of `exp` over the whole row, and on the workbench we only ever see part of the row. The trick is to keep running totals: the biggest score so far, the sum of `exp` so far, and the weighted sum of values so far. When a new tile turns up a bigger score, the old totals are rescaled by `exp(old max − new max)` and the new tile is added in.

Softmax in Tiles

Here it is for one row of twelve scores, in three tiles of four, next to the ordinary softmax:

```python
import torch
torch.manual_seed(0)
s = torch.randn(12) * 3      # one row of scores
v = torch.randn(12, 4)       # the values they weight

m = torch.tensor(float('-inf'))   # biggest score so far
total = torch.tensor(0.0)         # running sum of exp
out = torch.zeros(4)              # running weighted sum
for i in range(0, 12, 4):         # three tiles of 4
    t, vt = s[i:i+4], v[i:i+4]
    m_new = torch.maximum(m, t.max())
    fix = torch.exp(m - m_new)    # rescale old totals
    p = torch.exp(t - m_new)
    total = total * fix + p.sum()
    out = out * fix + p @ vt
    m = m_new

print(out / total)
print(torch.softmax(s, dim=0) @ v)
```

OUTPUT

```
tensor([-0.7028,  0.3768,  0.6610,  1.0891])
tensor([-0.7028,  0.3768,  0.6610,  1.0891])
```

Three tiles of four scores give exactly what the softmax over all twelve does. (Subtracting the biggest score before taking `exp` keeps the numbers from overflowing, which real implementations need anyway.)

Attention in Today's Models

Very few new models use plain multi-head attention any more. Most mix the variants in this chapter, and run all of them with flash attention:

Variant	Saves	Used in
Grouped-query	cache (shared heads)	Llama 3, Qwen3, irishQwen
Latent (MLA)	cache (compressed)	DeepSeek V3, Kimi K2
Sliding window	cache (fewer tokens)	Gemma 3, OLMo 3
Sparse (DSA)	work (fewer tokens)	DeepSeek V3.2, GLM-5
Hybrid (Gated DeltaNet)	cache (fixed state)	Qwen3-Next, Qwen3.5

With the cache under control, attention isn't the expensive part of a block any more. In each of Qwen3-0.6B's layers, the attention weights are `6,291,456` numbers and the feed-forward network's are `9,437,184`, `60%` of the layer, and every token goes through every one of them.

A model with more knowledge needs a bigger feed-forward network, which makes every token more expensive. Unless, of course, each token only had to use part of it.